

Venus PendlePTVaultAdapter Contract Audit Report

Please Note

1. The analysis of the Severity is purely based on the smart contracts mentioned in the Audit Scope and does not include any other potential contracts deployed by the Owner. No applications or operations were reviewed for Severity. No product code has been reviewed.
2. Due to the time limit, the audit team did not do much in-depth research on the business logic of the project. It is more about discovering issues in the smart contracts themselves.

Audit Period: 2026/03/03 - 2026/03/06 (YYYY/MM/DD)

Overall Risk: **Medium**

Project Name	Venus PendlePTVaultAdapter Contract Audit
Github	https://github.com/VenusProtocol/venus-periphery/pull/53
Background	Hello @nevi_0x , thank you for your quick response, attached is the context for the audit scope: https://www.notion.so/Audit-Scope-PendlePTVaultAdapter-2026-03-03-317b4e5771bb8061bd85e3bca248b8c8?source=copy_link • Timeline wise, any earliest availability to start the audit would work for us

Smart contracts list:

No.	File Name	Link	Verdict	Details
1	PendlePTVaultAdapter.sol	https://github.com/VenusProtocol/venus-periphery/blob/feat/vpd-613/contracts/pendle-pt-fixed-rate-vault/PendlePTVaultAdapter.sol	Medium	[M01] [M02] [L01] [L02] [L03] [I01] [I02] [I03]
2	IPendlePTVaultAdapter.sol	https://github.com/VenusProtocol/venus-periphery/blob/feat/vpd-613/contracts/pendle-pt-fixed-rate-vault/interfaces/IPendlePTVaultAdapter.sol	Green	
3	IVenusComptroller.sol	https://github.com/VenusProtocol/venus-periphery/blob/feat/vpd-613/contracts/pendle-pt-fixed-rate-vault/interfaces/IVenusComptroller.sol	Green	
4	IVenusVToken.sol	https://github.com/VenusProtocol/venus-periphery/blob/feat/vpd-613/contracts/pendle-pt-fixed-rate-vault/interfaces/IVenusVToken.sol	Green	
5	019-deploy-pendle-pt-vault-adapter.ts	https://github.com/VenusProtocol/venus-periphery/blob/feat/vpd-613/deploy/019-deploy-pendle-pt-vault-adapter.ts	Green	
6	test-pendlePTVaultAdapter	https://github.com/VenusProtocol/venus-periphery/tree/feat/vpd-613/tests/hardhat/Fork/pendlePTVaultAdapter	Green	

Findings

[M01] PT Tokens Can Be Stolen by Anyone

Contract PendlePTVaultAdapter.sol

Severity Level Medium

Description

Description

In the deposit function of the contract, it accepts any token to deposit for vTokens. The swap of anyToken => PT token is handled by PendleRouter. Inside the function, it uses the inline `_swapToPt` function to help users to swap the tokens.

After the swap, the function relies on the return `netPtOut` value to determine how many PT tokens it receives.

So here introduces a bug, the `netPtOut` returned by `_swapToPt` function is not reliable, it can be faked by constructing malicious `input.limit`

When swap happens in PendleRouter, it will check if the user use limit order to complete the swap, if so, **`input.limit.limitRouter`** will be used to fill user order and use the return value as the final swap amount, which finally return as `netPtOut` to the PendlePTVaultAdapter contract.

Attack scenario:

1. User call deposit function by indicating `input.tokenIn` as a legitimate whitelisted token (e.g. `slisBNB`), which successfully passes both the adapter's received amount validation and the SY contract's token whitelist check.
2. User also indicating **`limit.limitRouter` as a self-controlled contract** to return fake `netPtOut`
3. The PendlePTVaultAdapter use the fake `netPtOut` to deposit PT Token to Venus Pool using its own PT token balance

Vulnerable Code

JavaScript

```
// =====Step1 deposit
function deposit(
    address pendleMarket,
    uint256 minPtOut,
    ApproxParams calldata guessPtOut,
    TokenInput calldata input,
    LimitOrderData calldata limit
) external whenNotPaused nonReentrant
onlyRegisteredMarket(pendleMarket) returns (uint256 netVTokensMinted)
{
    if (input.netTokenIn == 0) revert ZeroAmount();
```

```

        if (input.tokenIn == address(0)) revert InvalidTokenInput();

        uint256 netPtOut;
        {
            MarketConfig storage config = markets[pendleMarket];
            address pt = config.pt;
            address vToken = config.vToken;
            uint256 ptBalanceBefore =
                IERC20(pt).balanceOf(address(this));

            //@audit Fake input.tokenIn controlled by user, which is
            used to bypass the received validation.
            uint256 balanceBefore =
                IERC20(input.tokenIn).balanceOf(address(this));
            IERC20(input.tokenIn).safeTransferFrom(msg.sender, address(this),
                input.netTokenIn);
            uint256 received =
                IERC20(input.tokenIn).balanceOf(address(this)) - balanceBefore;

            if (input.netTokenIn != received) revert
                InputAmountMismatch(input.netTokenIn, received);
            //@audit swaps happens here
            netPtOut = _swapToPt(pendleMarket, minPtOut, guessPtOut,
                input, limit);
            //@===Step4: fake netPtOut is used, use contract owned Pt
            to deposit.
            netVTokensMinted = _mintVTokens(pt, vToken, netPtOut);

            _sweepDust(pt, msg.sender, ptBalanceBefore);
            _sweepDust(input.tokenIn, msg.sender, balanceBefore);
        }

        emit Deposited(pendleMarket, msg.sender, input.tokenIn,
            input.netTokenIn, netPtOut, netVTokensMinted);
    }

    //@===Step2 SY token swap in Pendle router
    function _mintSyFromToken(address receiver, address SY, uint256
        minSyOut, TokenInput calldata inp)
        internal
        returns (uint256 netSyOut)
    {
        SwapType swapType = inp.swapData.swapType;

        uint256 netTokenMintSy;

        if (swapType == SwapType.NONE) {

```

```

        // @audit Transfer input.tokenIn into the router(must be
        whitelist)
        _transferIn(inp.tokenIn, msg.sender, inp.netTokenIn);
        netTokenMintSy = inp.netTokenIn;
    } else if (swapType == SwapType.ETH_WETH) {
        _transferIn(inp.tokenIn, msg.sender, inp.netTokenIn);
        _wrap_unwrap_ETH(inp.tokenIn, inp.tokenMintSy,
inp.netTokenIn);
        netTokenMintSy = inp.netTokenIn;
    } else {
        _swapTokenInput(inp);
        netTokenMintSy = _selfBalance(inp.tokenMintSy);
    }

    netSyOut = __mintSy(receiver, SY, netTokenMintSy, minSyOut,
inp);
}

function _swapTokenInput(TokenInput calldata inp) internal {
    if (inp.tokenIn == NATIVE) _transferIn(NATIVE, msg.sender,
inp.netTokenIn);
    else _transferFrom(IERC20(inp.tokenIn), msg.sender,
inp.pendleSwap, inp.netTokenIn);

    IPSwapAggregator(inp.pendleSwap).swap{value: inp.tokenIn ==
NATIVE ? inp.netTokenIn : 0}({
        inp.tokenIn, inp.netTokenIn, inp.swapData
    });
}

//===Step3 Pt token swap in Pendle Router
function _swapExactSyForPt(
    address receiver,
    address market,
    uint256 exactSyIn,
    uint256 minPtOut,
    ApproxParams calldata guessPtOut,
    LimitOrderData calldata limit
) internal returns (uint256 netPtOut, uint256 netSyFee) {
    (IStandardizedYield SY,, IPYieldToken YT) =
IPMarket(market).readTokens();
    uint256 netSyLeft = exactSyIn;
    bool doMarketOrder = true;

    if (!_isEmptyLimit(limit)) {
        // @audit Since lim.limitRouter is controlled by user, all
        return value can be fake, espically the netPtOut
        (netSyLeft, netPtOut, netSyFee, doMarketOrder) =
_fillLimit(receiver, SY, netSyLeft, limit);
        if (doMarketOrder) {

```

```

        _transferOut(address(SY), market, netSyLeft);
    } else {
        _transferOut(address(SY), receiver, netSyLeft);
    }
}

if (doMarketOrder) {
    (uint256 netPtOutMarket,) =

_readMarket(market).approxSwapExactSyForPtV2(YT.newIndex(), netSyLeft,
block.timestamp, guessPtOut);

    (, uint256 netSyFeeMarket) =
IPMarket(market).swapSyForExactPt(receiver, netPtOutMarket,
EMPTY_BYTES);

    netPtOut += netPtOutMarket;
    netSyFee += netSyFeeMarket;
}

if (netPtOut < minPtOut) revert("Slippage:
INSUFFICIENT_PT_OUT");
}

```

Impact:

The PT tokens remaining in Adapter will be transferred by anyone.

Additionally, the depositNative function is affected by the same logical vulnerability.

Recommendation	Use Before-After balance validation pattern to determine the received PT token amount.
-----------------------	--

Status	Fixed in f9089ae . The <code>_swapToPt()</code> and <code>_swapToPtNative()</code> functions no longer return or rely on the value reported by the Pendle Router. The return value of <code>swapExactTokenForPt()</code> is now explicitly discarded, and <code>netPtOut</code> is instead derived from the PT balance delta of the adapter contract (<code>IERC20(pt).balanceOf(address(this)) - ptBalanceBefore</code>). This ensures that regardless of any value a caller-supplied <code>limitRouter</code> may report, only the PT amount actually credited to the adapter is used for subsequent Venus minting, eliminating the inflation vector.
---------------	---

```

JavaScript
_swapToPt(pendleMarket, minPtOut, guessPtOut, input,
limit);

```

```
netPtOut = IERC20(pt).balanceOf(address(this)) -  
ptBalanceBefore;  
  
// 3. Deposit PT into Venus - vTokens go to user  
netVTokensMinted = _mintVTokens(pt, vToken, netPtOut);
```

[M02] Residual SY tokens can be stranded when limit orders skip the AMM

Contract PendlePTVaultAdapter.sol

Severity Level Medium

Description

Description

In the Pendle router contract, there is an existing [epsSkipMarket](#) input that is a threshold ratio inside LimitOrderData that determines whether the Pendle router should send leftover SY to the AMM after limit orders have partially filled. It is essentially a gas optimization where if the limit orders consumed almost all the SY, the remaining dust isn't worth an AMM swap.

In the `_swapExactSyForPt` function, when LimitOrderData is non-empty originating from the PendlePTVaultAdapter, the sophisticated swap path is taken. Inside `_swapExactSyForPt`:

```
JavaScript  
  
// ActionBase.sol - _swapExactSyForPt  
if (!_isEmptyLimit(limit)) {  
    (netSyLeft, netPtOut, netSyFee, doMarketOrder) =  
    _fillLimit(receiver, SY, netSyLeft,  
    limit);  
    if (doMarketOrder) {  
        _transferOut(address(SY), market, netSyLeft); // remaining  
SY → AMM  
    } else {  
        _transferOut(address(SY), receiver, netSyLeft); //  
remaining SY → receiver  
    }  
}
```

When limit orders fill enough volume that the remainder is below the `epsSkipMarket` threshold, `doMarketOrder = false` and residual SY tokens are sent to the receiver.

JavaScript

```
doMarketOrder = netLeft > netInput.mulDown(lim.epsSkipMarket);
```

As shown above, in the adapter's `_swapToPt`, the receiver is `address(this)` i.e. the `PendlePTVaultAdapter`, which is inputted within the `_swapToPt/_swapToPtNative` function:

JavaScript

```
(netPtOut, , ) =  
IPAllActionV3(PENDLE_ROUTER).swapExactTokenForPt(  
    address(this), // PT receiver = adapter (so we can deposit  
    into Venus)  
  
    (netPtOut, , ) =  
    IPAllActionV3(PENDLE_ROUTER).swapExactTokenForPt{ value: msg.value }(  
        address(this),
```

Back in the `PendlePTVaultAdapter` `deposit/depositNative` functions, it only sweeps `pt` and `input.tokenIn`. `SY` tokens are never swept. The `SY` residual becomes permanently stuck in the adapter, recoverable only via `sweepTokens` (`onlyOwner` function). As such, the user loses the value of the residual `SY` (although as mentioned above, the impact is limited). This is not an issue if the user interacts with the `pendle router` directly, since they can specify the receiver address directly.

JavaScript

```
_sweepDust(pt, msg.sender, ptBalanceBefore);  
_sweepDust(input.tokenIn, msg.sender, balanceBefore);  
  
    // 3. Safety sweep: not expected with exact-in swap, but  
    guards against  
    //    unexpected Router behavior leaving residual PT in the  
    adapter  
    _sweepDust(pt, msg.sender, ptBalanceBefore);  
  
    // 4. Safety refund: not expected with exact-in swap, but  
    guards against  
    //    unexpected native BNB returned to the adapter during the  
    swap  
    _refundNativeDust(nativeBalanceBefore);
```

Vulnerable Code

This impacts the deposit/depositNative functions, where a swap from SY to PT token is required, and only when a specific limit order is utilized.

Impact

User loses residual SY value when limit orders trigger the epsSkipMarket threshold, as the netPtOut returned to the adapter only reflects PT tokens received from limit fills, not the unswapped SY remainder (minPtOut slippage must still be met).

Recommendation	Consider calling <code>_sweepDust</code> with the corresponding sy token as well within the respective deposit functions.
-----------------------	---

Status	Fixed in 6be9a24 . Both <code>deposit()</code> and <code>depositNative()</code> now capture a <code>syBalanceBefore</code> snapshot of the adapter's SY token balance prior to the swap, and invoke <code>_sweepDust(config.sy, msg.sender, syBalanceBefore)</code> after the swap completes. This ensures that any residual SY tokens left in the adapter — which can occur when limit orders trigger Pendle's <code>epsSkipMarket</code> threshold causing partial fills — are returned to the caller rather than remaining stranded in the contract.
---------------	---

```
JavaScript
_sweepDust(config.sy, msg.sender, syBalanceBefore);
```

[L01] Expired market can be used in deposit

Contract	PendlePTVaultAdapter.sol
-----------------	--------------------------

Severity Level	Low
-----------------------	-----

Description	The <code>deposit()</code> and <code>depositNative()</code> functions do not include an explicit <code>beforeMaturity</code> check, relying instead on the Pendle Router to naturally revert post-maturity swaps. This assumption holds for normal usage paths — the official Pendle Limit Router enforces its own expiry check in <code>_validateSkipSigAndFilterOrders</code> :
--------------------	---

```
JavaScript
require(block.timestamp < IPYieldToken(YT).expiry(), "LOP: PY expired");
```

However, as described in [M01], the adapter does not validate `limit.limitRouter` against a whitelist, allowing a caller to supply a self-controlled contract in its place. A malicious `limitRouter` has no expiry check, and when `doMarketOrder = false` (limit

orders fully fill), the AMM path — which contains the market-level expiry check in `IPMarket(market).swapSyForExactPt` — is bypassed entirely:

```
JavaScript
if (doMarketOrder) {
    // expiry check is inside swapSyForExactPt
    (, uint256 netSyFeeMarket) = IPMarket(market).swapSyForExactPt(...);
}
// doMarketOrder = false → AMM skipped → no expiry check
```

This means an attacker who has already deployed a malicious `limitRouter` (as in [M01]) can additionally deposit into an expired market through the adapter, bypassing the expected post-maturity rejection.

Note that this finding depends on the malicious `limitRouter` prerequisite from [M01] — it is not exploitable in isolation with the official Pendle infrastructure.

Vulnerable Code

```
JavaScript
function deposit(
    address pendleMarket,
    uint256 minPtOut,
    ApproxParams calldata guessPtOut,
    TokenInput calldata input,
    LimitOrderData calldata limit
) external whenNotPaused nonReentrant
//@audit missing beforeMaturity modifier
onlyRegisteredMarket(pendleMarket) returns (uint256 netVTokensMinted)

function depositNative(
    address pendleMarket,
    uint256 minPtOut,
    ApproxParams calldata guessPtOut,
    TokenInput calldata input,
    LimitOrderData calldata limit
)
    external
    payable
    whenNotPaused
    nonReentrant
    onlyRegisteredMarket(pendleMarket)
    returns (uint256 netVTokensMinted)
//@audit missing beforeMaturity modifier

function _swapExactSyForPt(
    address receiver,
    address market,
```

```

        uint256 exactSyIn,
        uint256 minPtOut,
        ApproxParams calldata guessPtOut,
        LimitOrderData calldata limit
    ) internal returns (uint256 netPtOut, uint256 netSyFee) {
        (IStandardizedYield SY,, IPYieldToken YT) =
        IPMarket(market).readTokens();
        uint256 netSyLeft = exactSyIn;
        bool doMarketOrder = true;

        if (!_isEmptyLimit(limit)) {
            (netSyLeft, netPtOut, netSyFee, doMarketOrder) =
            _fillLimit(receiver, SY, netSyLeft, limit);
            if (doMarketOrder) {
                _transferOut(address(SY), market, netSyLeft);
            } else {
                _transferOut(address(SY), receiver, netSyLeft);
            }
        }

        if (doMarketOrder) {
            (uint256 netPtOutMarket,) =

            _readMarket(market).approxSwapExactSyForPtV2(YT.newIndex(), netSyLeft,
            block.timestamp, guessPtOut);
            // @audit Expiration check applies here, if doMarketOrder == false,
            // bypass the check
            (, uint256 netSyFeeMarket) =
            IPMarket(market).swapSyForExactPt(receiver, netPtOutMarket,
            EMPTY_BYTES);

            netPtOut += netPtOutMarket;
            netSyFee += netSyFeeMarket;
        }

        if (netPtOut < minPtOut) revert("Slippage:
        INSUFFICIENT_PT_OUT");
    }

```

Impact:

An attacker exploiting [M01] can additionally deposit into an expired Pendle market through the adapter. If Venus has active incentive programs at the time, this may allow the attacker to earn protocol rewards using expired PT tokens that should no longer be eligible.

Recommendation Add the beforeMaturity modifier to both deposit() and depositNative() as a defense-in-depth measure.

Status	Acknowledged. The team noted that deposit() and depositNative() intentionally omit a local beforeMaturity guard. The Pendle Router natively reverts any swapExactTokenForPt call submitted after a market's expiry, providing an equivalent and sufficient enforcement at the protocol level. Adding a redundant on-chain check in the adapter would increase gas cost without improving security or user protection.
---------------	---

[L02] Missing check of expiration time when add market

Contract	PendlePTVaultAdapter.sol
Severity Level	Low

Description

Description

In the addMarket function, the approved caller can add the market to the whitelist for user deposit later, but the function fails to check if the PT token of the market is already expired, which make expired PT can also be added to the market whitelist.

Vulnerable Code

```
JavaScript
function addMarket(address pendleMarket, address vToken) external {
    _checkAccessAllowed("addMarket(address,address)");
    if (pendleMarket == address(0)) revert ZeroAddress();
    if (vToken == address(0)) revert ZeroAddress();
    if (markets[pendleMarket].pt != address(0)) revert
    MarketAlreadyRegistered(pendleMarket);

    // Validate that the vToken is listed in the Comptroller
    if (!IVenusComptroller(COMPTROLLER).isMarketListed(vToken))
    revert VTokenNotListed(vToken);

    // Derive token addresses and maturity from Pendle market
    contract
    // solhint-disable-next-line var-name-mixedcase
    (IStandardizedYield _SY, IPPrincipalToken _PT, IPYieldToken
    _YT) = IPMarket(pendleMarket).readTokens();

    // Validate that the vToken's underlying matches the derived
    PT
    if (IVenusVToken(vToken).underlying() != address(_PT)) revert
    UnderlyingMismatch(vToken, address(_PT));

    uint256 maturity = IPMarket(pendleMarket).expiry();
    //@audit missing check if PT is already expired
    markets[pendleMarket] = MarketConfig({
        pt: address(_PT),
        sy: address(_SY),
```

```

        yt: address(_YT),
        vToken: vToken,
        maturity: maturity
    });

    marketList.push(pendleMarket);

    emit MarketAdded(pendleMarket, address(_PT), vToken,
    maturity);
}

```

Impact:

Expired market can be added to the market whitelist.

Recommendation Add validation of the PT expiration time.

Status Fixed in [1977854](#). A maturity validation check has been added to addMarket(). The function now reverts with MarketAlreadyMatured if the provided Pendle market's expiry timestamp is at or before the current block.timestamp at the time of registration. This prevents already-expired PT markets from being registered, which would otherwise result in a market where withdraw() is permanently inaccessible due to the beforeMaturity guard, leaving users unable to exit their positions before maturity.

```

JavaScript
if (!(block.timestamp < maturity)) revert
MarketAlreadyMatured(maturity, block.timestamp);

```

[L03] Missing check of minPtOut in deposit

Contract PendlePTVaultAdapter.sol

Severity Level **Low**

Description **Description**
In deposit and depositNative function, minPtOut is used to protect from slippage loss, but the function fails to validate if minPtOut is greater than 0, which makes the user may suffer loss.

Vulnerable Code

JavaScript

```
function deposit(
    address pendleMarket,
    uint256 minPtOut,
    ApproxParams calldata guessPtOut,
    TokenInput calldata input,
    LimitOrderData calldata limit
) external whenNotPaused nonReentrant
onlyRegisteredMarket(pendleMarket) returns (uint256 netVTokensMinted)
{
    if (input.netTokenIn == 0) revert ZeroAmount();
    if (input.tokenIn == address(0)) revert InvalidTokenInput();

    uint256 netPtOut;
    {
        MarketConfig storage config = markets[pendleMarket];
        address pt = config.pt;
        address vToken = config.vToken;
        uint256 ptBalanceBefore =
IERC20(pt).balanceOf(address(this));

        // 1. Pull tokens from user → adapter (accepts any token
from Pendle's tokensIn)
        uint256 balanceBefore =
IERC20(input.tokenIn).balanceOf(address(this));
IERC20(input.tokenIn).safeTransferFrom(msg.sender,
address(this), input.netTokenIn);
        uint256 received =
IERC20(input.tokenIn).balanceOf(address(this)) - balanceBefore;

        // Validate actual received amount matches Pendle's
expected input
        if (input.netTokenIn != received) revert
InputAmountMismatch(input.netTokenIn, received);

        // 2. Swap tokenIn → PT via Pendle Router (Pendle handles
aggregator routing if needed)
        netPtOut = _swapToPt(pendleMarket, minPtOut, guessPtOut,
input, limit);

        // 3. Deposit PT into Venus – vTokens go to user
        netVTokensMinted = _mintVTokens(pt, vToken, netPtOut);

        // 4. Safety sweep: not expected with exact-in swap, but
guards against
        //     unexpected Router/token behavior leaving residual
tokens in the adapter
        _sweepDust(pt, msg.sender, ptBalanceBefore);
        _sweepDust(input.tokenIn, msg.sender, balanceBefore);
    }
}
```

```
        emit Deposited(pendleMarket, msg.sender, input.tokenIn,
input.netTokenIn, netPtOut, netVTokensMinted);
    }

    /// @inheritdoc IPendlePTVaultAdapter
    function depositNative(
        address pendleMarket,
        uint256 minPtOut,
        ApproxParams calldata guessPtOut,
        TokenInput calldata input,
        LimitOrderData calldata limit
    )
        external
        payable
        whenNotPaused
        nonReentrant
        onlyRegisteredMarket(pendleMarket)
        returns (uint256 netVTokensMinted)
    {
        if (msg.value == 0) revert ZeroAmount();

        MarketConfig storage config = markets[pendleMarket];
        address pt = config.pt;
        address vToken = config.vToken;
        uint256 ptBalanceBefore = IERC20(pt).balanceOf(address(this));
        uint256 nativeBalanceBefore = address(this).balance -
msg.value;

        // Validate calldata consistency
        if (input.netTokenIn != msg.value) revert
InputAmountMismatch(input.netTokenIn, msg.value);

        // 1. Swap native BNB → PT via Pendle Router
        uint256 netPtOut = _swapToPtNative(pendleMarket, minPtOut,
guessPtOut, input, limit);

        // 2. Deposit PT into Venus – vTokens go to user
        netVTokensMinted = _mintVTokens(pt, vToken, netPtOut);

        // 3. Safety sweep: not expected with exact-in swap, but
guards against
        //     unexpected Router behavior leaving residual PT in the
adapter
        _sweepDust(pt, msg.sender, ptBalanceBefore);

        // 4. Safety refund: not expected with exact-in swap, but
guards against
```

```

swap    //    unexpected native BNB returned to the adapter during the
        _refundNativeDust(nativeBalanceBefore);

        emit Deposited(pendleMarket, msg.sender, input.tokenIn,
msg.value, netPtOut, netVTokensMinted);
    }

```

Impact:

Users may swap Pt with no slippage protection.

Recommendation Add validation of minPtOut.

Status Acknowledged. minPtOut is a standard caller-supplied slippage parameter passed directly to the Pendle Router, which enforces the minimum output constraint internally. Imposing an additional minPtOut > 0 requirement at the adapter level would be an opinionated restriction that could break compatibility with aggregator integrations — aggregators commonly pass minPtOut = 0 and rely on their own slippage logic upstream. The responsibility for setting an appropriate minPtOut is therefore delegated to the caller.

[I01] Missing Native Token Validation for input.tokenIn in depositNative() Function

Contract PendlePTVaultAdapter.sol

Severity Level **Informational**

Description **Description**
The depositNative() function accepts native BNB via msg.value and forwards it to the Pendle Router to swap for PT tokens. However, the function does not validate that input.tokenIn is set to address(0) (the conventional representation of native currency in Pendle's TokenHelper).

Vulnerable Code

JavaScript

```

function depositNative(
    address pendleMarket,
    uint256 minPtOut,
    ApproxParams calldata guessPtOut,

```

```

        TokenInput calldata input,
        LimitOrderData calldata limit
    )
    external
    payable
    whenNotPaused
    nonReentrant
    onlyRegisteredMarket(pendleMarket)
    returns (uint256 netVTokensMinted)
{
    if (msg.value == 0) revert ZeroAmount();

    MarketConfig storage config = markets[pendleMarket];
    address pt = config.pt;
    address vToken = config.vToken;
    uint256 ptBalanceBefore = IERC20(pt).balanceOf(address(this));
    uint256 nativeBalanceBefore = address(this).balance -
msg.value;

    // Validate calldata consistency
    if (input.netTokenIn != msg.value) revert
InputAmountMismatch(input.netTokenIn, msg.value);

    // 1. Swap native BNB → PT via Pendle Router
    uint256 netPtOut = _swapToPtNative(pendleMarket, minPtOut,
guessPtOut, input, limit);

    // 2. Deposit PT into Venus – vTokens go to user
    netVTokensMinted = _mintVTokens(pt, vToken, netPtOut);

    // 3. Safety sweep: not expected with exact-in swap, but
guards against
    //     unexpected Router behavior leaving residual PT in the
adapter
    _sweepDust(pt, msg.sender, ptBalanceBefore);

    // 4. Safety refund: not expected with exact-in swap, but
guards against
    //     unexpected native BNB returned to the adapter during the
swap
    _refundNativeDust(nativeBalanceBefore);
    emit Deposited(pendleMarket, msg.sender, input.tokenIn,
msg.value, netPtOut, netVTokensMinted);
}

function _swapToPtNative(
    address pendleMarket,
    uint256 minPtOut,

```

```

        ApproxParams calldata guessPtOut,
        TokenInput calldata input,
        LimitOrderData calldata limit
    ) internal returns (uint256 netPtOut) {
        (netPtOut, , ) =
        IPAllActionV3(PENDLE_ROUTER).swapExactTokenForPt{ value: msg.value }(
            address(this),
            pendleMarket,
            minPtOut,
            guessPtOut,
            input,
            limit
        );
    }

```

Impact

There is no loss of funds. The transaction will always revert before any state change occurs, and the native BNB is returned to the caller.

However, the following quality concerns apply:

- Poor error surfacing: The revert originates deep inside the Pendle Router with a generic SafeERC20 error rather than a clear, protocol-level error at the adapter boundary.
- Wasted gas: The caller bears unnecessary gas costs before the revert is triggered.

Recommendation	Add an explicit check at the entry point of depositNative() to enforce that input.tokenIn is the native currency address.
-----------------------	---

Status	Fixed in ada7f3f . Add a check to ensure that <code>input.tokenIn</code> must be the zero address.
---------------	--

```

JavaScript
if (input.tokenIn != address(0)) revert InvalidTokenInput();

```

[I02] Missing Events in Emergency Sweep Functions

Contract	PendlePTVaultAdapter.sol
-----------------	--------------------------

Severity Level**Informational****Description**

The sweepTokens and sweepNative functions transfer funds from the contract but do not emit events. This reduces transparency and makes it difficult to track emergency fund recovery operations, which is particularly important for governance oversight and auditing.

JavaScript

```
function sweepTokens(address token, address to, uint256 amount) external
onlyOwner {
    if (token == address(0)) revert ZeroAddress();
    if (to == address(0)) revert ZeroAddress();
    IERC20(token).safeTransfer(to, amount);
    // ❌ No event emitted
}

function sweepNative(address payable to, uint256 amount) external
onlyOwner {
    if (to == address(0)) revert ZeroAddress();
    Address.sendValue(to, amount);
    // ❌ No event emitted
}
```

Recommendation Add explicit events for sweep operations:

JavaScript

```
// Add event definitions
event TokensSwept(address indexed token, address indexed to, uint256
amount);
event NativeSwept(address indexed to, uint256 amount);
function sweepTokens(address token, address to, uint256 amount) external
onlyOwner {
    if (token == address(0)) revert ZeroAddress();
    if (to == address(0)) revert ZeroAddress();
    IERC20(token).safeTransfer(to, amount);
    emit TokensSwept(token, to, amount); // ✅ Add event
}

function sweepNative(address payable to, uint256 amount) external
onlyOwner {
    if (to == address(0)) revert ZeroAddress();
    Address.sendValue(to, amount);
    emit NativeSwept(to, amount); // ✅ Add event
}
```

Status

Fixed in [36a4279](#). Events have been added for these two functions.

JavaScript

```
function sweepTokens(address token, address to, uint256 amount)
external onlyOwner {
    if (token == address(0)) revert ZeroAddress();
    if (to == address(0)) revert ZeroAddress();
    IERC20(token).safeTransfer(to, amount);
    emit SweepTokens(token, to, amount);
}

/// @inheritdoc IPendlePTVaultAdapter
function sweepNative(address payable to, uint256 amount) external
onlyOwner {
    if (to == address(0)) revert ZeroAddress();
    Address.sendValue(to, amount);
    emit SweepNative(to, amount);
}
```

[I03] Double Negation Logic Reduces Code Readability**Contract**

PendlePTVaultAdapter.sol

Severity Level

Informational

Description

Multiple locations in the contract use double negation patterns (e.g., `!(a < b)` instead of `a >= b`), which reduces code readability and increases cognitive load for reviewers and auditors. Direct positive logic is clearer and easier to understand.

JavaScript

```
function isMatured(address pendleMarket) external view returns (bool) {
    if (markets[pendleMarket].pt == address(0)) revert
    MarketNotRegistered(pendleMarket);
    return !(block.timestamp < markets[pendleMarket].maturity); // ✗
    Double negation
}

modifier beforeMaturity(address pendleMarket) {
    uint256 maturity = markets[pendleMarket].maturity;
    if (!(block.timestamp < maturity)) revert
    MarketAlreadyMatured(maturity, block.timestamp); // ✗ Double negation
}
```

```
    -;  
}
```

Recommendation Replace all double negation patterns with direct positive logic:

```
JavaScript  
function isMatured(address pendleMarket) external view returns (bool) {  
    MarketConfig storage config = markets[pendleMarket]; // Cache  
storage access  
    if (config.pt == address(0)) revert  
MarketNotRegistered(pendleMarket);  
    return block.timestamp >= config.maturity; // ✓ Direct positive  
logic  
}  
  
modifier beforeMaturity(address pendleMarket) {  
    uint256 maturity = markets[pendleMarket].maturity;  
    if (block.timestamp >= maturity) revert  
MarketAlreadyMatured(maturity, block.timestamp); // ✓ Direct positive  
logic  
    -;  
}
```

Status Acknowledged. The development team confirmed that the double-negation form `!(block.timestamp < maturity)` is intentionally used in place of the equivalent `block.timestamp >= maturity` to suppress a solhint linter warning. Both expressions are semantically identical and compile to the same bytecode with no gas difference. No code change is required.
